

Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Кафедра “Системи автоматизованого проектування”

ЗВІТ
про проходження проектно-технологічної практики
студента IV курсу групи ПП-44
Верещака БОГДАНА

База практики _____
ФОП Ціп Микола Богданович,
(назва та адреса підприємства)
Львівська область, смт. Куликів, вул. Шевченка, 50

Термін практики з «20» квітня 2026 р. по з «09» травня 2026 р.

Керівники практики

від кафедри _____ асист. кафедри САП Іванина В. В.
(підпис) (наук.ст, вч.звання, прізвище, ім'я, по батькові)

від бази практики _____ ФОП Ціп Микола Богданович
(підпис) (посада, прізвище, ім'я, по батькові)

Тема БКР: “Розроблення системи динамічної адаптації складності рівнів у ігровому середовищі”

Керівник БКР _____ асист. каф. САП Патерега Юрій Ігорович
(підпис) (Посада, Прізвище, ініціали)

ОЦІНКА _____

Дата _____

Зміст

ВСТУП.....	3
Актуальність роботи.....	3
Мета виконаної роботи	3
Завдання, які виконані в роботі.....	3
Коротка характеристика розділів звіту	4
РОЗДІЛ 1	5
Опис бази практики	5
Огляд літературних джерел	5
Характеристика предметної області	5
Характеристика середовищ, технологій, фреймворків та засобів розроблення системи	6
Обґрунтування теми бакалаврської кваліфікаційної роботи	8
Актуальність теми	8
Загальна схема	8
Короткий опис	8
Методи реалізації	9
Висновки до розділу.....	9
РОЗДІЛ 2	10
Розроблення вимог до системи та її компонентів	10
Технологічні вимоги до інформаційної системи	10
Функціональні вимоги до інформаційної системи.....	10
Розроблення схем системи.....	11
Функціональна схема.....	11
Структурна схема.....	13
Інфологічна схема	13
Опис бізнес логіки	14
Реалізація частини системи	15
Опис вхідних даних для системи та вимог до результатів її роботи.....	19
Опис контрольних прикладів та засобів тестування для підтвердження працездатності системи	19
Висновки до розділу.....	20
ВИСНОВОК.....	21
Список використаних літературних джерел.....	22

ВСТУП

Актуальність роботи

Сучасна ігрова індустрія стикається з нетривіальною проблемою: статична складність у комп'ютерних іграх не відповідає різноманітності гравців. Навіть якщо розробник пропонує кілька фіксованих рівнів складності, жоден із них не враховує реального стану конкретного гравця в конкретний момент часу. Гравці мають різний досвід, швидкість реакції і темп навчання, а їхні можливості можуть змінюватися навіть протягом однієї сесії. За деякими оцінками, близько 40% гравців залишають гру через невідповідність складності їхнім навичкам, що безпосередньо впливає на комерційний успіх і репутацію продукту.

Системи динамічної адаптації складності (Dynamic Difficulty Adjustment, DDA) виникли як відповідь на цю проблему. Вони аналізують поведінкові метрики гравця в реальному часі і непомітно коригують параметри ігрового світу, утримуючи гравця у стані «потoku» – оптимального балансу між складністю завдання і рівнем навичок. DDA вже має понад двадцятирічну дослідницьку традицію, проте більшість реалізацій обмежуються або примітивними евристичними правилами, або вузькоспеціалізованими ML-підходами, не адаптованими до широкого класу ігор.

Дана робота є безпосереднім продовженням попередньої практики, в якій було створено прототип системи збору поведінкових метрик гравця та базовий ігровий прототип на Unity. На цьому етапі проект перейшов до активної фази: реалізована перша версія евристичної DDA-системи, переосмислена архітектура ігрового контролера на основі 3D-моделі персонажа, розширена аудіосистема, додано освітлення, UI та закладено основу для подальшого порівняльного дослідження різних алгоритмів адаптації. Тема є актуальною як для наукового дослідження, так і для практичного застосування у реальних проектах ігрової індустрії.

Мета виконаної роботи

Метою роботи є розроблення та дослідження системи динамічної адаптації складності рівнів у 3D-платформері на базі Unity Engine. Система повинна в реальному часі аналізувати просторові та часові поведінкові метрики гравця, а на їх основі автоматично коригувати параметри ігрового середовища таким чином, щоб підтримувати оптимальний баланс виклику і навичок. У рамках практики пріоритетом є реалізація евристичної DDA-системи та підготовка архітектурного фундаменту для майбутнього порівняльного аналізу з алгоритмами на основі Machine Learning.

Завдання, які виконані в роботі

У ході практики виконано такі завдання.

- Рефакторинг і розширення системи реєстрації смертей: виправлення логічних помилок у координатах позицій смертей, ізоляція статистики між

рівнями, уніфікація часових і просторових даних через структуру DeathRecord.

- Реалізація евристичного DDA-менеджера: алгоритм кластеризації смертей, розрахунок центроїда проблемної зони, зважений вплив на перешкоди залежно від відстані до центру кластера.
- Розробка базового класу DynamicObstacleBase та рефакторинг усіх типів перешкод (SimpleDynamicPlatform, MovingPlatform, FallingPlatform, SpikyPlatform) під єдиний інтерфейс IDynamicObstacle.
- Реалізація логіки ускладнення геймплею при домінуванні гравця: тригерний механізм на основі чекпоінтів, налаштовувані ліміти Min/Max та індивідуальна вага впливу (Weight) для кожної перешкоди.
- Перехід від 2D-спрайту до 3D-персонажа: переписаний PlayerMovement, оновлений PlayerTracker з логікою збереження останньої безпечної позиції (LastSafePosition), підтримка анімацій через Animator.
- Реалізація системи плавного переходу (TransitionManager) для непомітного застосування змін DDA під затемнення екрану.
- Розширення системи подій GameEvents, розроблення ScriptableObject-архітектури для аудіосистеми (AudioEvent, MusicTrack, SurfaceData, SurfaceLibrary, SurfaceAudioMap).
- Підготовка рівня: левел-дизайн сцени з різними типами перешкод, освітлення, базовий UI (лічильник смертей, час проходження).

Коротка характеристика розділів звіту

Перший розділ звіту містить стислий опис бази практики, огляд предметної області та технологічного стеку, а також обґрунтування теми бакалаврської кваліфікаційної роботи. Основна увага приділяється концептуальним засадам DDA і порівняльній характеристиці інструментів, які використовуються в проєкті.

Другий розділ присвячений технічній стороні практики: формалізованим вимогам до системи, її функціональним та структурним схемам, опису вхідних і вихідних даних, а також результатам тестування реалізованих механік. У цьому розділі детально розбирається архітектура DDA і спосіб взаємодії її компонентів між собою.

РОЗДІЛ 1

Опис бази практики

Базою практики є ФОП Ціп Микола Богданович, що спеціалізується на розробці програмного забезпечення у сфері Full-Stack та інформаційних систем. Практична діяльність підприємця пов'язана з роботою з великими масивами даних: їх збором, обробкою та аналізом для прийняття обґрунтованих бізнес-рішень. Зокрема, він залучений до проектів, що активно використовують аналіз поведінкових даних користувачів. Таке середовище виявилось органічним для розробки системи збору та аналізу ігрової телеметрії, оскільки в основі обох завдань лежить схожий цикл: збір поведінкових сигналів, їх структурування та перетворення на дієві рішення.

Крім безпосередньої розробки, база практики надала можливість поглибити розуміння принципів побудови інформаційних систем, зокрема у частині проектування архітектури збору даних і забезпечення модульності коду. Підприємець також має досвід у методичній роботі зі студентами, що дозволило ефективніше опрацьовувати складніші аспекти реалізації.

Огляд літературних джерел

Характеристика предметної області

Предметна область роботи охоплює геймдизайн і ігрову аналітику, з центральним фокусом на проблематиці динамічної адаптації складності. Складність є одним із найбільш критичних параметрів ігрового балансу: занадто проста гра викликає нудьгу, надмірно складна – фрустрацію та відтік аудиторії. Оптимальний стан гравця в теорії ігрового дизайну прийнято описувати через концепцію «поток» (flow), сформульовану психологом Міхаєм Чіксентміхаї: гравець перебуває у стані максимальної залученості, коли складність завдань відповідає його поточному рівню навичок.

Традиційні підходи до балансування складності ґрунтуються на заздалегідь визначених кривих, розроблених досвідченими геймдизайнерами. Однак вони мають принципове обмеження: різноманітність гравців настільки велика за досвідом, темпом навчання і стилем гри, що єдина крива складності не може задовольнити всіх. Більше того, навіть для одного гравця уподобання щодо складності змінюються з часом і залежать від контексту.

Динамічна корекція складності (DDA) вирішує ці проблеми, використовуючи поведінкові дані в реальному часі. Ключовим джерелом сигналів є просторова телеметрія: координати смертей, маршрути переміщення, позиції стрибків. На їх основі DDA-система формує «проблемні зони» (trouble zones) – ділянки рівня, де гравець систематично зазнає невдачі. Зона визначається через кластеризацію координат смертей: якщо в певному радіусі за короткий проміжок часу зафіксовано кілька смертей, система ідентифікує центроїд кластера і застосовує вплив на найближчі до нього перешкоди з урахуванням відстані до центра.

Поряд зі смертями, важливими метриками є час бездіяльності гравця після поразки (idle time) та успішність взаємодії з конкретними перешкодами (obstacle success rate). Ці показники дозволяють розрізнити ситуацію, коли гравець розгублений і на межі виходу з гри, від ситуації легкої помилки після якої гравець без труднощів рухається далі.

Сучасний стан дослідницької сфери DDA охоплює декілька парадигм. Класичні евристичні методи покладаються на набір правил, визначених вручну (якщо гравець помирає більше N разів на одному місці – полегшити). Machine Learning-підходи, зокрема навчання з підкріпленням (Reinforcement Learning) і ансамблеві методи (Random Forest), дозволяють агенту самостійно формувати правила адаптації на основі накопиченого досвіду. У даному проекті планується послідовно реалізувати і порівняти ці підходи: статичну базову гру як контрольну групу, евристичну DDA (поточний етап), Random Forest і Reinforcement Learning.

Характеристика середовищ, технологій, фреймворків та засобів розроблення системи

Розробка проекту ведеться на ігровому рушії Unity у поєднанні з мовою програмування C#. Unity обрано через широку екосистему, кросплатформність і доступність безкоштовного плану для некомерційних проектів. Рушій надає готові підсистеми фізики, рендерингу, анімацій та аудіо, що дозволяє зосередити зусилля на реалізації дослідницької частини, а не базової інфраструктури.

C# використовується як основна мова написання ігрової логіки. Статична типізація, підтримка об'єктно-орієнтованого та частково функціонального програмування, а також тісна інтеграція з Unity роблять її оптимальним вибором для реалізації складних архітектурних патернів. У проекті активно застосовуються принципи ООП: інкапсуляція в класах компонентів, успадкування через базовий клас DynamicObstacleBase, поліморфізм через інтерфейс IDynamicObstacle.

Серед пакетів Unity, що активно використовуються в проекті, ключову роль відіграють Input System (гнучке та масштабоване управління вводом), Cinemachine (система віртуальних камер з автоматичними переходами) і ProBuilder (інструмент для прототипування геометрії рівнів безпосередньо у редакторі). Система контролю версій базується на Unity Version Control, що забезпечує збереження повної історії змін. Для написання скриптів використовується Visual Studio Code з розширенням для Unity.

Аудіоархітектура побудована на ScriptableObject-патерні: замість жорстко закодованих посилань на кліпи кожен тип звукової події (AudioEvent) є незалежним ресурсом з налаштовуваними параметрами гучності та висоти тону. SurfaceLibrary зберігає прив'язку між фізичним матеріалом поверхні і відповідним набором звуків, що дозволяє легко розширювати кількість типів поверхонь.

Для підготовки ML-компонентів у майбутніх етапах планується використання Python з бібліотеками scikit-learn і pandas для навчання моделей Random Forest, а також пакети Unity ML-Agents для реалізації Reinforcement

Learning безпосередньо в рушії. Навчені моделі будуть інтегровані в Unity через формат ONNX за допомогою пакету Unity Sentis.

Технологія	Призначення	Переваги	Недоліки
Unity	Ігровий рушій	Крос-платформність, велика спільнота, безкоштовний план	Витрата ресурсів, крива навчання
C#	Мова програмування	Статична типізація, інтеграція з Unity	Вимагає більшого часу на написання коду, гірша продуктивність
Input System	Управління вводом	Підтримка різних пристроїв, гнучкість	Складніше у вивченні ніж старий Input Manager
Unity Version Control	Система контролю версій	Інтеграція з Unity, зручна робота в команді, збереження історії змін	Обмежений безкоштовний функціонал, залежність від інтернет-з'єднання
ProBuilder	Прототипування рівнів	Швидкі зміни геометрії в редакторі	Обмежений функціонал порівняно з 3D-редакторами
Input System	Створення контролера персонажа	Підтримка різних пристроїв введення, гнучкість налаштувань	Більш складний у вивченні, ніж старий Input Manager
ScriptableObject (Audio)	Аудіоархітектура	Модульність, повторне використання, легка розширюваність	Потребує акуратного управління посиленнями
Unity ML-Agents	Reinforcement Learning	Глибока інтеграція з рушієм, відкритий код	Тривале навчання, потреба у Headless-режимі
Newtonsoft Json	Робота з JSON	Висока продуктивність, стандарт де-факто в екосистемі .NET	Додаткова залежність у проекті
Unity Sentis / ONNX	Інференс ML-моделей	Крос-платформний запуск навчених моделей	Підтримка обмеженого підмножини операторів
Unity Version Control	Контроль версій	Інтеграція з Unity, повна історія змін	Обмежений безкоштовний функціонал

Табл. 1. Порівняльна характеристика технологій розробки

Обґрунтування теми бакалаврської кваліфікаційної роботи

Тема: "Розроблення системи динамічної адаптації складності рівнів у ігровому середовищі"

Актуальність теми

Актуальність теми обумовлена невирішеністю проблеми балансування ігрового досвіду для гетерогенної аудиторії. Попри понад двадцятирічну історію DDA-досліджень, переважна більшість комерційних реалізацій або обмежується примітивними правилами, або є власницькими і не підлягає аналізу. Відкрите дослідження ефективності різних підходів до DDA, з можливістю відтворення і верифікації результатів, представляє наукову цінність.

Практична цінність роботи полягає в тому, що проект є повноцінним функціональним прототипом, а не теоретичним дослідженням. Евристична DDA-система вже реалізована і перебуває на стадії налаштування. Вона здійснює кластеризацію координат смертей, обчислює центроїд проблемної зони і застосовує зважений вплив на параметри перешкод у радіусі адаптації. Ця реалізація слугує базовою лінією для порівняння з наступними підходами.

У подальшому планується реалізація алгоритму на основі Random Forest: модель буде навчена на даних, зібраних системою аналітики під час реальних ігрових сесій, і зможе прогнозувати оптимальний множник складності без жорстко заданих правил. На фінальному етапі буде розроблено Reinforcement Learning-агента («Режисер»), для навчання якого передбачається використання агента-гравця (бота). Ця концепція відповідає науковому підходу Asymmetric Self-Play і дозволяє проводити мільйони симуляцій у прискореному режимі.

Реалізація потребує поєднання знань у сферах геймдизайну, алгоритмів кластеризації, машинного навчання та архітектурного проектування програмного забезпечення. Доступний технологічний стек (Unity, C#, Python, scikit-learn, ML-Agents) повністю покриває всі потреби проекту.

Загальна схема

У проєкті реалізовано базовий замкнений цикл DDA. Під час гри відбувається збір подій і метрик сесії, далі формується проблемна зона на основі повторюваних смертей і просторового аналізу з урахуванням останньої безпечної позиції. Після цього система знаходить динамічні перешкоди в межах заданого радіуса впливу та змінює їх параметри в обмежених межах. Зміни застосовуються так, щоб не ламати геймплей і не бути помітними для гравця. Для цього використовується підхід із затемненням екрану, під час якого перешкоди можуть змінити розміри або таймінги без візуального “стрибка”.

Короткий опис

На поточному етапі реалізовано евристичний DDA, який виконує як полегшення, так і ускладнення. Полегшення активується, коли система виявляє

кластер повторюваних смертей у певній зоні. Ускладнення активується, коли гравець проходить секцію без смертей. Обидва механізми працюють локально, тобто змінюють не всю гру, а конкретні перешкоди в потрібній частині рівня. Це дозволяє зберегти загальну структуру рівня та не знецінювати навички гравця.

У межах практики також закладено основу для подальшої наукової частини диплома. Наступними кроками визначено підготовку даних для Random Forest (побудова ознак, навчання на зібраній телеметрії) і розгляд Reinforcement Learning як підходу до керування складністю через агента, який оптимізує “стан потоку” на основі нагород і штрафів.

Методи реалізації

Методична основа реалізації включає:

1. збір телеметрії у вигляді подій і агрегованих метрик,
2. просторовий аналіз проблемних зон,
3. евристичні правила зміни параметрів перешкод у допустимих межах,
4. непомітне застосування змін через керований екранний перехід,
5. підготовку архітектури до подальшої заміни евристики на ML-моделі.

Висновки до розділу

У розділі проведено аналіз предметної області систем DDA в контексті ігрової індустрії. Визначено ключові поведінкові метрики, що лежать в основі адаптації: просторові координати смертей, час бездіяльності гравця, успішність взаємодії з перешкодами. Розглянуто основні парадигми реалізації DDA – евристичні методи, Random Forest і Reinforcement Learning – та обґрунтовано послідовний порівняльний підхід, що є науковим внеском роботи.

Проведено огляд і порівняльну характеристику технологічного стеку проекту. Показано, що Unity у поєднанні з C#, ScriptableObject-архітектурою для аудіо, ML-Agents і ONNX/Sentis утворює повноцінну платформу для реалізації всіх запланованих етапів дослідження.

РОЗДІЛ 2

Розроблення вимог до системи та її компонентів

Технологічні вимоги до інформаційної системи

Розроблювана система повинна відповідати сучасним стандартам розробки ігрових додатків та забезпечувати високу продуктивність під час обчислення метрик. Кросплатформність гарантується використанням рушія Unity, що дозволяє компілювати проект під ОС Windows, Linux, а також у форматі WebGL-застосунку.

Ігрове середовище реалізується у вигляді 3D-платформера з камерою від третьої особи. Візуальна складова базується на Universal Render Pipeline (URP), що вимагає підтримки сучасних шейдерів для реалізації хоррор-елементів (динамічне освітлення, туман, постобробка). На відміну від попередніх ітерацій з мінімалістичним оформленням, поточна версія вимагає використання оптимізованих 3D-моделей та матеріалів, які взаємодіють із фізичною підсистемою просторового звуку (ScriptableObjects).

Ігрові механіки вимагають наявності повноцінного 3D-контролера персонажа з урахуванням фізичної інерції, збереження моментуму та додаткових механік для покращення "відчуття гри". На рівнях мають бути різні типи платформ і пасток (рухомі платформи, платформи що падають, обертові шипи, прості динамічні платформи). Вони повинні формувати відрізки з підвищеною складністю, щоб система DDA мала реальні дані для аналізу. Має бути реалізована смерть персонажа (падіння або зіткнення з небезпечними об'єктами), відродження на активованому чекпоїнті, а також перехід на наступний рівень після виконання умов.

До підсистеми збору даних та DDA (Dynamic Difficulty Adjustment) висувуються жорсткі вимоги щодо оптимізації. Аналітика та розрахунок евристичних формул не повинні викликати падіння кадрової частоти (FPS) нижче 60 кадрів на секунду у цільовому середовищі. Це досягається завдяки асинхронному запису JSON-даних та використанню патерну Registry для управління пулом перешкод без витратних пошукових запитів в ієрархії сцени.

Код повинен бути структурований з використанням ООП і SOLID. Система має підтримувати додавання нових перешкод і метрик без зміни основи. Архітектура DDA повинна дозволити заміну евристики на ML-підхід у наступних етапах (Random Forest, Reinforcement Learning), використовуючи однаковий канал керування перешкодами.

Функціональні вимоги до інформаційної системи

Ігрове середовище. Гравець керує 3D-персонажем у режимі платформера за допомогою клавіатури. Рух включає переміщення в усіх горизонтальних напрямках, стрибок із регульованою висотою залежно від тривалості натискання, а також Coyote Time (короткий буфер стрибка після сходу з краю) і Jump Buffering (запам'ятовування натискання пробілу перед приземленням).

Модель персонажа плавно повертається у напрямку руху за допомогою Quaternion.Slerp.

Ігрова механіка. Рівень містить не менше чотирьох типів інтерактивних перешкод: платформи, що падають, рухомі платформи, шипові платформи, а також статичні елементи. Кожен тип перешкоди має конфігуровані параметри Min/Max для обмеження меж адаптації та індивідуальну вагу впливу (Weight), доступну для редагування в інспекторі Unity. Гравець може помирати (падіння, зіткнення з небезпечними об'єктами) і відроджуватися на останньому активованому чекпоінті. Перехід між рівнями можливий після досягнення цілі.

Система DDA. DDA-менеджер відстежує кластери смертей у часовому вікні (deathWindow). Якщо кількість смертей у кластері досягає порогу, система обчислює центроїд проблемної зони і застосовує полегшення до перешкод у радіусі адаптації (adaptationRadius) з урахуванням зваженого впливу залежно від відстані до центра. Зміни параметрів перешкод відбуваються під час плавного затемнення екрану (TransitionManager), щоб залишитися непомітними для гравця. Якщо гравець проходить відрізок між двома чекпоінтами без жодної смерті, ініціюється ускладнення перешкод у зоні попереду.

Система аналітики. AnalyticsManager автоматично ініціалізується при запуску і веде лог поточної сесії у форматі NDJSON. GameSession є синглтоном, що зберігається між сценами і відстежує всі ключові ігрові події: старт/кінець рівня, смерті, чекпоінти, повторні спроби, взаємодії з перешкодами, активність вводу, бездіяльність. При зміні рівня стан смертей очищається. При виході з гри система форсовано скидає буфер на диск.

Аудіосистема. Звуки кроків, стрибків, приземлення і ударів прив'язані до типу поверхні через SurfaceLibrary. Кожна подія AudioEvent містить масив кліпів і діапазони гучності та висоти тону для рандомізації відтворення, що забезпечує природне звучання без повторень.

Інтерфейс користувача. Гравець бачить лічильник смертей і час проходження поточного рівня. Плавний перехід (fade-in/fade-out) використовується для прихованого застосування змін DDA.

Розроблення схем системи

Функціональна схема

Функціональна схема показує повний цикл від збору даних до застосування DDA. Логіка роботи має включати такі етапи: ініціалізація аналітики, реєстрація подій сесії та рівня, накопичення метрик, виявлення проблемної зони (death cluster), вибір перешкод у радіусі, підготовка зміни параметрів і застосування її у правильний момент (затемнення екрану при респаві). Також схема повинна відображати гілку ускладнення, яка активується після проходження секції без смертей.

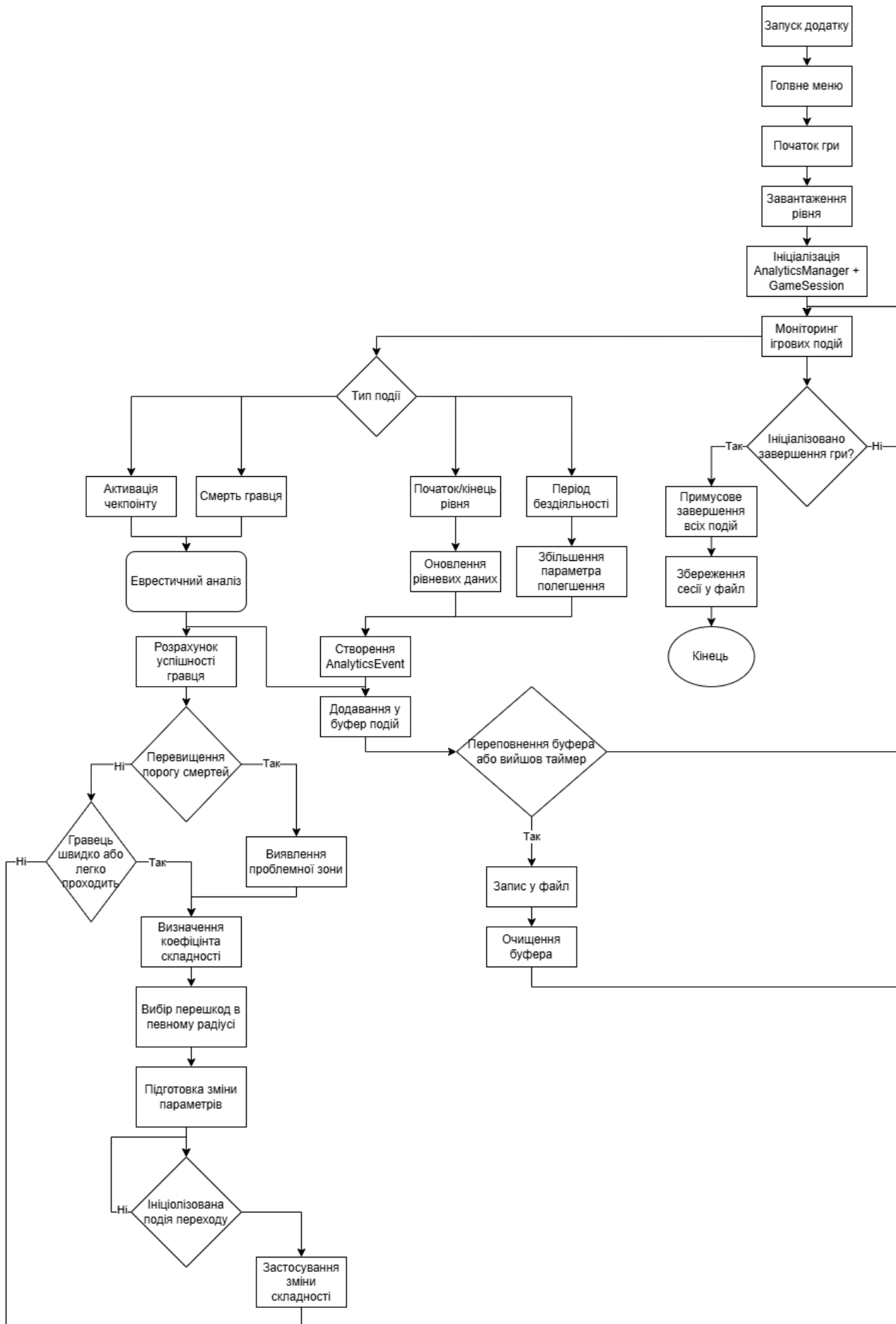


Рис. 1. Функціональна схема оброблення подій для динамічної зміни складності

Структурна схема

Структурна схема демонструє компонентну організацію системи та взаємозв'язки між ключовими класами. На верхньому рівні знаходиться DDAManager, що підписується на події шини GameEvents і керує реєстром перешкод (Registry Pattern). PlayerTracker і GameSession утворюють аналітичний контур: перший збирає просторові дані, другий агрегує ігрові події і формує метрики. Усі типи перешкод (FallingPlatform, MovingPlatform, SpikyPlatform, SimpleDynamicPlatform) успадковують DynamicObstacleBase, який реалізує IDynamicObstacle і надає єдиний метод ApplyDDA(float effectiveMultiplier). AnalyticsManager є незалежним компонентом, що отримує події через GameSession і асинхронно записує їх на диск. TransitionManager як модуль екранного переходу, який дає безпечний момент для застосування змін. Також наявні допоміжні системи (респаун, UI, аудіо, камера), які забезпечують повноцінний ігровий цикл.

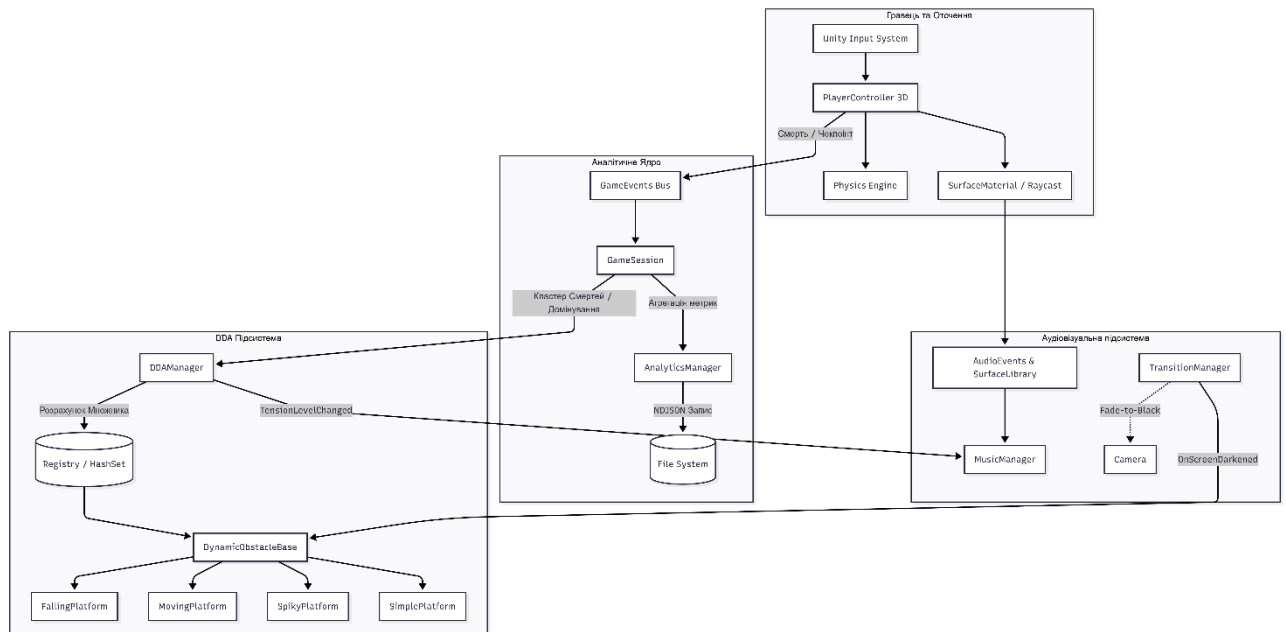


Рис. 2. Структурна схема, що описує організацію системи між ключовими класами

Інфологічна схема

Інфологічна схема відображає структуру даних, якими оперує система. Центальною сутністю є *DeathRecord*, що об'єднує координату смерті (*Vector3*) і часову мітку (*float*) в єдину структуру. Це усуває попередню проблему розсинхронізації окремих масивів позицій і часу. Сутність *SESSION* містить унікальний ідентифікатор, час початку і масив *ANALYTICS_EVENT* – полів подій різних типів (*death*, *checkpoint*, *level_start* тощо), кожна з яких має власну схему додаткових даних у форматі *JSON*. *DynamicObstacleState* зберігає

поточний множник (currentScaleFactor або currentSpeed), початковий стан та ліміти Min/Max для кожного екземпляра перешкоди.

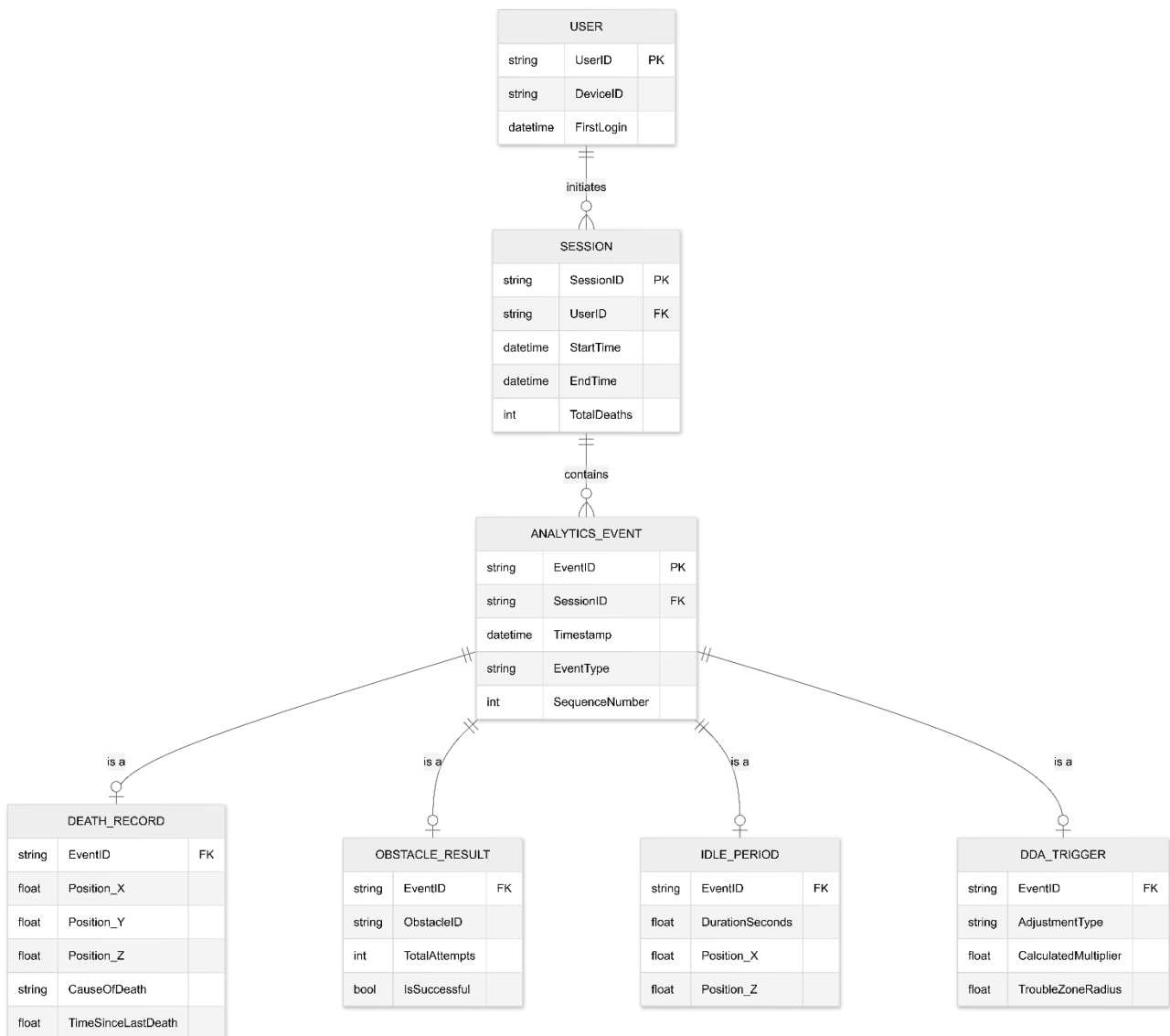


Рис. 3. Інфологічна схема, описує структуру даних, якими оперує система

Опис бізнес логіки

Бізнес-логіка системи будується навколо подієвого підходу. Після запуску гри аналітика ініціалізується автоматично, створюються ідентифікатори користувача та сесії, відкривається новий лог-файл. Паралельно запускається `GameSession`, який зберігається між сценами і відстежує ігрові події. Події перетворюються у структуровані записи та буферизуються для асинхронного запису без впливу на FPS.

При смерті гравця реєструється подія з причиною і позицією. Для просторової точності проблемна зона визначається не як “точка на kill plane”, а як точка, що враховує останню безпечну позицію перед падінням. `GameSession` накопичує події смертей у часовому вікні та перевіряє, чи утворився кластер. Якщо кластер виявлено, `DDA Manager` аналізує зареєстровані перешкоди поблизу центру кластера, визначає силу впливу відстанню та формує зміни

параметрів (полегшення). Щоб зміни не виглядали як “підміна світу”, вони застосовуються у момент затемнення екрану при респавні, що є технічною точкою синхронізації.

Окремо реалізовано ускладнення. Якщо гравець проходить відрізок рівня між чекпоїнтами без смертей, система фіксує домінування і передає сигнал DDA Manager. У такому випадку складність підвищується локально для перешкод попереду, щоб зберегти баланс “виклик-уміння” без різких стрибків складності.

Реалізація системи

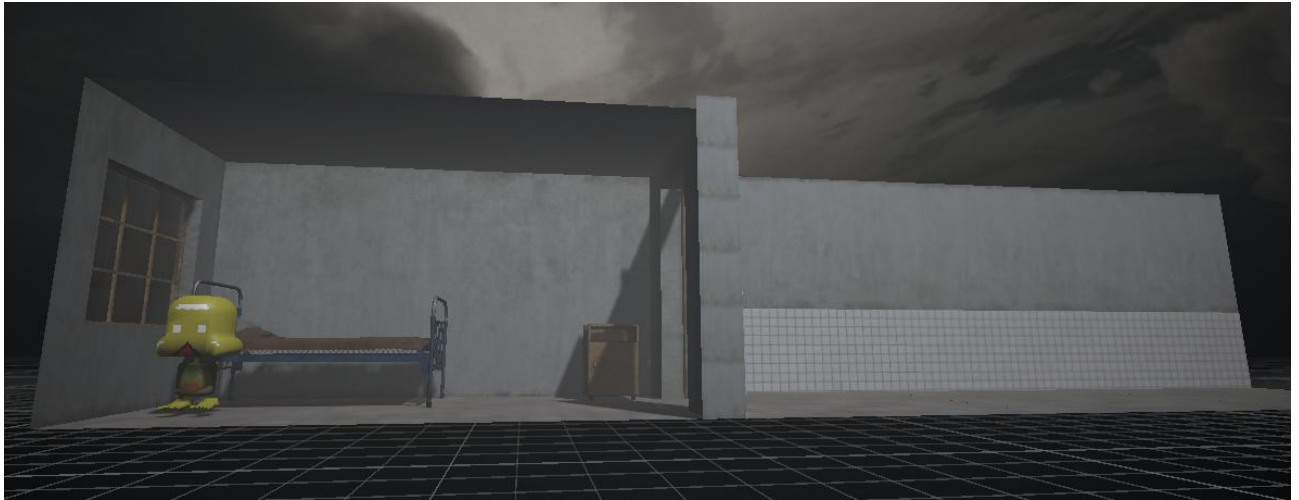


Рис. 4. Реалізація 3D-середовища в Unity з налаштованим URP та освітленням

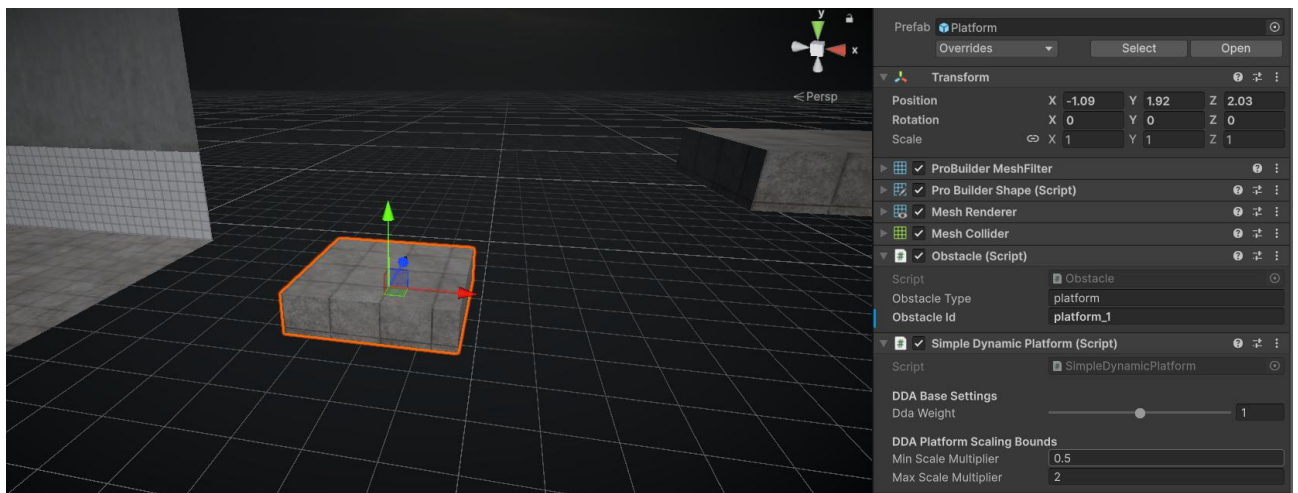


Рис. 5. Налаштування параметрів DDA (Min/Max Scale, DDA Weight) в інспекторі перешкоди SimpleDynamicPlatform

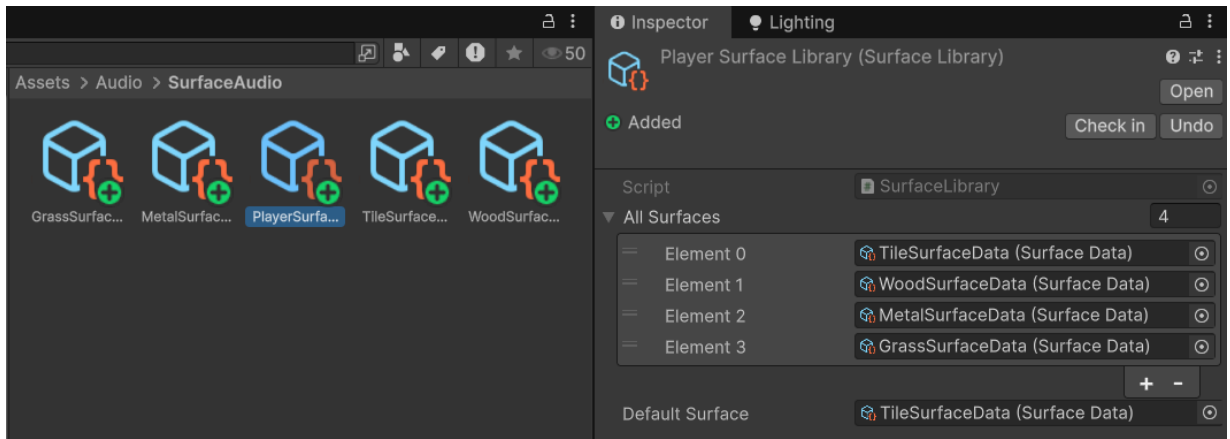


Рис. 6. Модульна система аудіо (SurfaceData, AudioEvent)

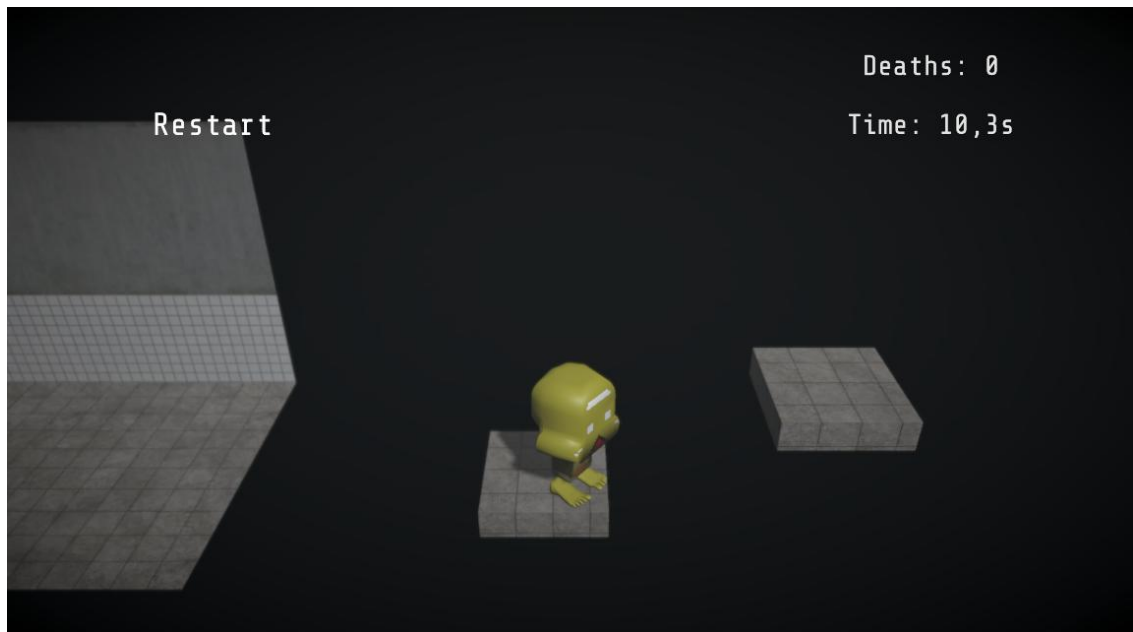


Рис. 7. Перевірка зміни складності. Без жодних змін

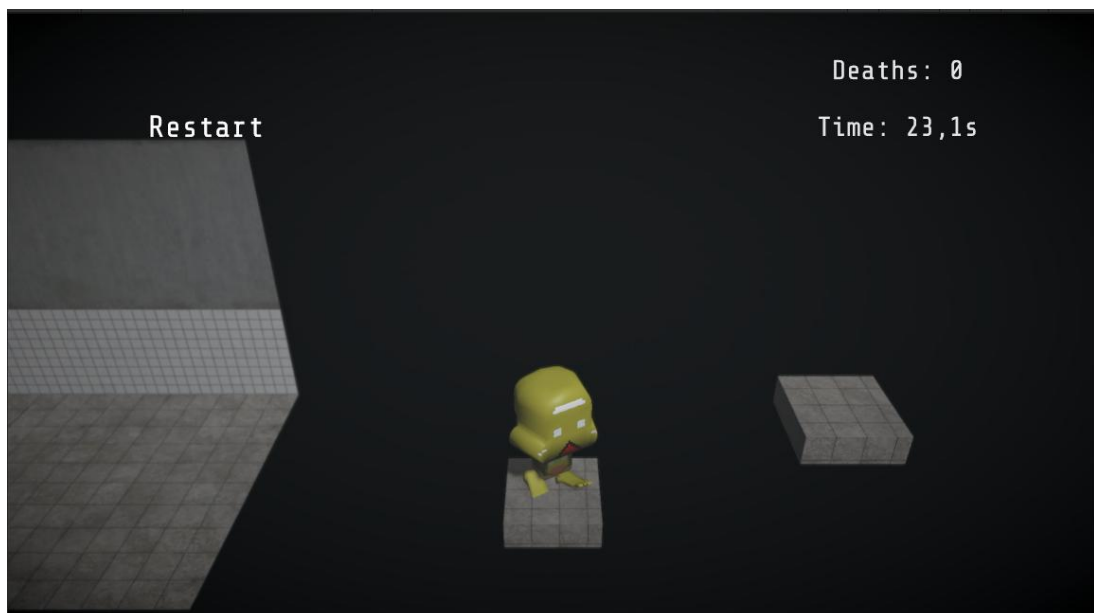


Рис. 8. Перевірка зміни складності. Ускладнення (гравець з легкістю проходить рівні)

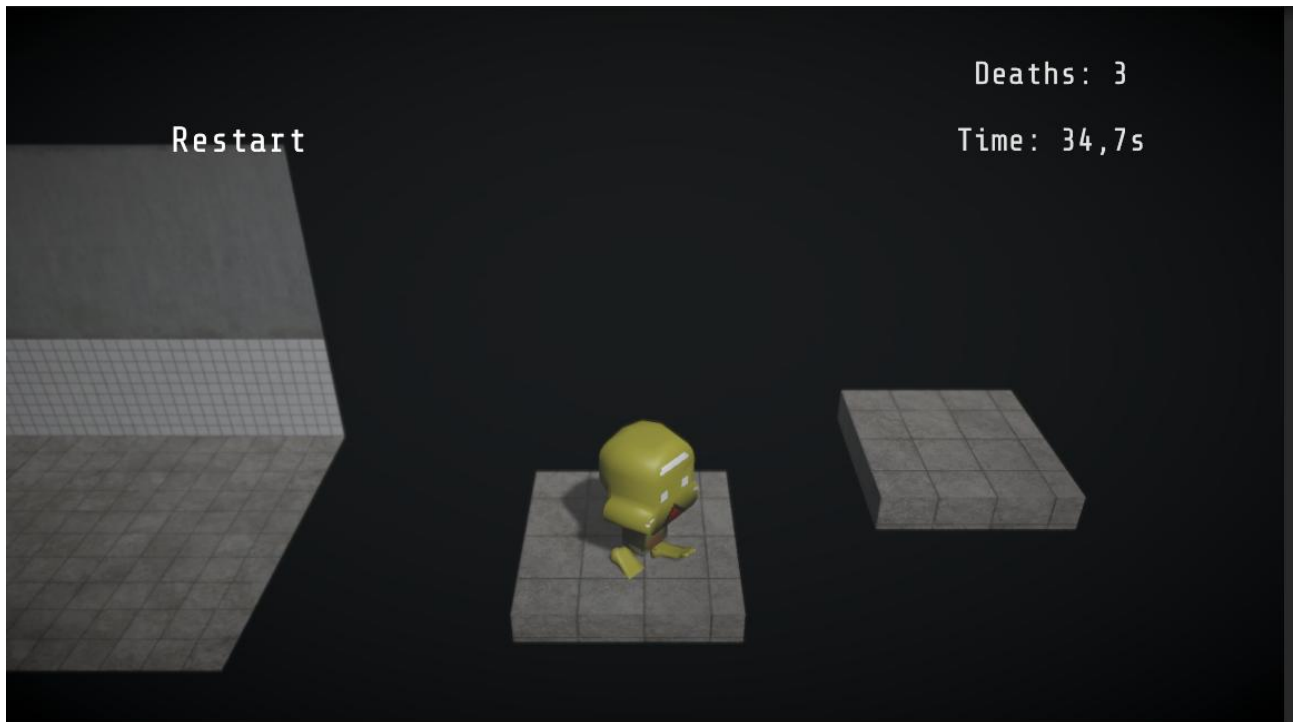


Рис. 9. Перевірка зміни складності. Полегшення (гравець вмирає в певній зоні)



Рис. 10. Сцена головного меню / UI



Рис. 11. Екран паузи під час гри / UI



Рис. 12. Екран налаштувань під час гри / UI

Tables > session_20260515_181423.ndjson

Filter Sort Columns Export

☐	T session_id	T user_id	T timestamp_iso	T event_type
☐	1d01eefca8f1477693008607...	907c05a2726d47629f2f1b5...	2026-05-15T15:14:23.99796...	session_start
☐	1d01eefca8f1477693008607...	907c05a2726d47629f2f1b5...	2026-05-15T15:14:24.43151...	app_resume
☐	1d01eefca8f1477693008607...	907c05a2726d47629f2f1b5...	2026-05-15T15:14:30.86190...	app_pause
☐	1d01eefca8f1477693008607...	907c05a2726d47629f2f1b5...	2026-05-15T15:14:37.415910...	app_resume
☐	1d01eefca8f1477693008607...	907c05a2726d47629f2f1b5...	2026-05-15T15:14:38.70840...	app_pause
☐	1d01eefca8f1477693008607...	907c05a2726d47629f2f1b5...	2026-05-15T15:20:24.96697...	app_resume
☐	1d01eefca8f1477693008607...	907c05a2726d47629f2f1b5...	2026-05-15T15:20:38.9527...	app_pause

Рис. 13. Структурований вид логів подій гри

Опис вхідних даних для системи та вимог до результатів її роботи

Вхідні дані

До вхідних даних системи належать: дії гравця (натискання клавіш, тривалість натискань, частота вводу у часовому вікні); ігрові події (старт і завершення рівня, смерть персонажа з координатами і причиною, активація чекпоінтів, взаємодія з перешкодами); позиційні дані (поточна позиція гравця і LastSafePosition для точного визначення місця кластера смертей); системні параметри (ідентифікатори сесії, рівня, гравця; часові мітки; розмір часового вікна для кластеризації).

Вимоги до результатів

Система DDA повинна: коректно ідентифікувати кластери смертей у просторі та часі; застосовувати зважений вплив з урахуванням відстані до центроїда; забезпечувати непомітність змін для гравця через механізм затемнення; захищати параметри перешкод від виходу за межі Min/Max.

Система аналітики повинна: записувати структуровані NDJSON-логи з часовими мітками і типами подій; підтримувати конвертацію у формат масиву JSON для подальшої обробки; зберігати дані навіть при аварійному завершенні; не знижувати FPS гри.

Усі типи перешкод мають коректно реагувати на ApplyDDA: платформи, що падають, збільшують час тремтіння; рухомі платформи зменшують швидкість; шипові платформи збільшують затримку обертання; прості платформи масштабуються по осях X і Z без зміни Y (щоб не порушити висоту стрибків).

Опис контрольних прикладів та засобів тестування для підтвердження працездатності системи

Працездатність системи перевірялася шляхом симуляції критичних ігрових сценаріїв у середовищі Unity Editor. Перший контрольний приклад тестував систему евристичного полегшення. Гравцем було навмисно імітовано серію з п'яти падінь з однієї обертової платформи у прірву (KillPlane). Спостерігалася реакція системи: GameSession успішно сформував кластер смертей, після чого TransitionManager здійснив затемнення екрану. Під час перевірки параметрів у вікні Inspector було зафіксовано, що координати X та Z платформи збільшилися на розрахований коефіцієнт 1.214, тоді як координата Y залишилася незмінною для збереження цілісності левел-дизайну.

Другий контрольний приклад стосувався інверсії DDA (ускладнення гри). Гравець безперешкодно проходив секцію платформ від одного чекпоінта до іншого. Система коректно зареєструвала відсутність записів смертей (No-Death Streak) і викликала подію OnPlayerDominating. Наступні перешкоди (які знаходилися поза зоною видимості гравця) автоматично зменшили свої розміри та збільшили швидкість руху без очікування затемнення екрану, що підтверджує правильну роботу прапорця миттєвого застосування. Додатково тестувалася система аудіо. Стрибки персонажа по об'єктах з різними PhysicsMaterial супроводжувалися коректними звуковими ефектами, які обиралися зі ScriptableObjects словника (SurfaceLibrary) із випадковою зміною висоти тону для уникнення монотонності. Усі дії безперервно логувалися у файл NDJSON без фіксації просадок кадрової частоти в профайлері Unity.

Висновки до розділу

У другому розділі сформульовано повний набір технологічних і функціональних вимог до системи, що охоплює ігрове середовище, DDA-підсистему, аналітику й аудіосистему. Розроблено функціональну, структурну та інфологічну схеми, що описують взаємодію компонентів і потоки даних. Детально розкрито бізнес-логіку системи: від моменту реєстрації смерті до прихованого застосування зміни параметрів перешкоди.

Описано вхідні дані системи і вимоги до результатів її роботи. Наведено контрольні приклади і засоби тестування, що підтвердили коректну роботу кластеризації, захисту параметрів від некоректних значень і механізму непомітного застосування змін. Реалізована евристична система повністю відповідає сформульованим вимогам і утворює надійний фундамент для наступних етапів дослідження.

ВИСНОВОК

У ході переддипломної практики виконано комплекс робіт, спрямованих на розвиток ігрового прототипу та реалізацію першої функціональної версії системи динамічної адаптації складності. Практика стала логічним продовженням попереднього етапу, в якому було закладено аналітичну основу проекту: систему збору метрик GameSession і AnalyticsManager.

На цьому етапі реалізовано повноцінну евристичну DDA-систему. Алгоритм кластеризації смертей коректно ідентифікує проблемні зони на основі структури DeathRecord, яка об'єднує просторові координати і часові мітки в єдиний запис. Зважений вплив на перешкоди з урахуванням відстані до центроїда кластера забезпечує природне і пропорційне полегшення, а захист від «ослаблення» попередніх адаптацій гарантує консистентний досвід. Механізм ускладнення при домінуванні гравця реалізований через подію OnPlayerDominating, прив'язану до проходження чекпоінтів без смертей.

Архітектурні рішення, прийняті під час практики, свідомо орієнтовані на масштабованість. Registry Pattern у DDAManager дозволяє замінити або доповнити евристичну логіку реалізацією на основі Random Forest чи Reinforcement Learning без будь-яких змін у клієнтському коді перешкод. Розроблена концепція двоагентного тренування (бот-гравець і бот-режисер) відповідає науковому підходу Asymmetric Self-Play і є основою для фінального етапу дослідження.

Крім DDA, суттєво розширено сам ігровий прототип: здійснено перехід до 3D-персонажа з підтримкою анімацій, реалізовано ScriptableObject-архітектуру аудіосистеми з прив'язкою звуків до типів поверхонь, вдосконалено освітлення й UI. Ці зміни роблять проект придатним для реальних плейтестів, результати яких стануть навчальними даними для ML-моделей.

Усі поставлені завдання практики виконано в повному обсязі. Отримані результати підтверджують технологічну можливість реалізації теми БКР і формують конкретну дорожню карту подальшого дослідження: стрес-тест евристичної системи, збір датасету для Random Forest, налаштування ML-Agents Pipeline і порівняльний аналіз усіх чотирьох підходів до DDA за метриками Completion Rate, Session Time і непомітності змін для гравця.

Список використаних літературних джерел

1. Офіційна документація Unity [Електронний ресурс] // Режим доступу: <https://docs.unity3d.com/Manual/index.html>
2. Документація по мові програмування C# [Електронний ресурс] // Режим доступу: <https://docs.microsoft.com/en-us/dotnet/csharp/>
3. Документація бібліотеки Newtonsoft.Json (Json.NET) [Електронний ресурс] // Режим доступу: <https://www.newtonsoft.com/json/help/html/Introduction.htm>
4. Документація Unity Universal Render Pipeline (URP) [Електронний ресурс] // Режим доступу: <https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@latest/>
5. Документація Unity ML-Agents Toolkit [Електронний ресурс] // Режим доступу: <https://github.com/Unity-Technologies/ml-agents/tree/main/docs>
6. Nacke, L. E., & Drachen, A. (2011). Towards a Framework of Player Experience Research. Proceedings of the Second International Workshop on Evaluating Player Experience in Games at FDG.
7. Hullett, K., & Whitehead, J. (2010). Design Patterns in FPS Levels. Proceedings of the Fifth International Conference on the Foundations of Digital Games.
8. Charles, D., & Black, M. (2004). Dynamic Player Modelling: A Framework for Player-Centered Digital Games. Proceedings of the International Conference on Computer Games: Artificial Intelligence, Design and Education.
9. John F. Kolen & Su Xue & Meng Wu & Navid Aghdaie (2017). New Mexico Institute of Mining and Technology. Dynamic Difficulty Adjustment for Maximized Engagement in Digital Games.
10. Zohaib, M. (2018). Dynamic Difficulty Adjustment (DDA) in Computer Games: A Review. IEEE International Conference on Advances in Computing, Communications and Informatics.
11. Nicholas Fisher & Arun K. Kulshreshth (2018). Dynamic Difficulty Adjustment in Games: Concepts, Techniques, and Applications